

COMP 551 Project 3: Odd-One-Out Image Classification with a Siamese CNN

Uday Parmar(261081860)

Shahmeer Sajid(261073334)

Danyal Valika(261036640)

Kaggle Team: Group#31

Abstract

We address the odd-one-out image classification task, where each input contains five grayscale images and the goal is to identify the image that does not share the common visual property of the others. We propose a Siamese convolutional neural network (CNN) that maps each image into a shared embedding space and predicts the outlier using a parameter-free rule based on average pairwise L2 distance. To improve generalization under the strict 25,000-parameter limit, we apply shape normalization through bounding-box cropping and resizing, and concatenate a Sobel edge map as a second input channel. The model is trained using cross-entropy loss on distance-based scores and optimized with Adam. At inference, test-time augmentation (TTA) further improves robustness. Our final model achieves **67.10%** public Kaggle accuracy with **24,718** trainable parameters, substantially outperforming the **20.9%** logistic regression baseline.

Introduction

The odd-one-out task is a five-class classification problem in which the correct label cannot be inferred from any image independently; instead, it depends on the relationship between the five images in the group. This makes the problem fundamentally relational. The assignment is additionally constrained by a small dataset and a strict budget of at most 25,000 trainable parameters, ruling out large pretrained models and making overfitting a central concern.

Visual inspection suggests that the hidden shared property is primarily *shape-based*, while position, scale, and orientation vary across images and are largely irrelevant. This motivates our approach: combine targeted preprocessing that removes nuisance variation with a compact Siamese CNN whose decision rule explicitly reflects the geometry of the odd-one-out task.

Data Processing

Dataset. The data consists of 3,000 training groups and 2,000 test groups, where each group contains five 32×32 grayscale images. The label is the index of the outlier image.

Normalization. Raw pixel values are scaled to $[0, 1]$. Each image is thresholded to identify foreground pixels, cropped to the bounding box of the shape with a small padding, and resized back to 32×32 . This removes irrelevant translation and scale variation.

Edge Augmentation. Because shape boundaries are more informative than texture, we compute a Sobel edge map and concatenate it with the normalized image as a second input channel.

Training Augmentation. Random horizontal and vertical flips are applied during training. These transformations preserve the label because the outlier is defined by shape category rather than orientation.

Validation Strategy. We train on the full training set and use the labeled public portion of the test set for model selection. The private test set remains unseen until final leaderboard evaluation.

Methods

Model Overview. We use a Siamese CNN that encodes each of the five images with shared weights and identifies the outlier using a parameter-free geometric rule in embedding space.

Shared Encoder. Each input image has size $2 \times 32 \times 32$ (normalized image + Sobel channel). The encoder consists of three convolutional blocks followed by a fully connected projection:

Layer	Output Size
Conv (2→16) + BN + ReLU + MaxPool	16 × 16 × 16
Conv (16→32) + BN + ReLU + MaxPool	32 × 8 × 8
Conv (32→16) + BN + ReLU + MaxPool	16 × 4 × 4
Flatten + FC (256→58) + BN + ReLU	58

Outlier Detection. Let $\{\mathbf{e}_0, \dots, \mathbf{e}_4\}$ be the five embeddings. For image i , we compute

$$d_i = \frac{1}{4} \sum_{j \neq i} \|\mathbf{e}_i - \mathbf{e}_j\|_2^2.$$

The predicted outlier is $\arg \max_i d_i$. This rule directly matches the task structure and avoids adding a learned classifier head that can overfit.

Training. The five distance scores are treated as logits and optimized with cross-entropy loss. We train using Adam with learning rate 10^{-3} , batch size 64, and early stopping with patience 150 for a maximum of 400 epochs. We observed that replacing Batch Normalization with Dropout (rate 0.3) caused training loss to remain at approximately $\log(5) = 1.609$, corresponding to random predictions, indicating that Dropout disrupted learning in this compact network.

Hyperparameters.

Parameter	Value
Learning Rate	1×10^{-3}
Optimizer	Adam
Batch Size	64
Max Epochs	400
Early Stopping Patience	150
Embedding Dimension	58
Augmentations	HFlip, VFlip
TTA Views	8

Parameter Breakdown.

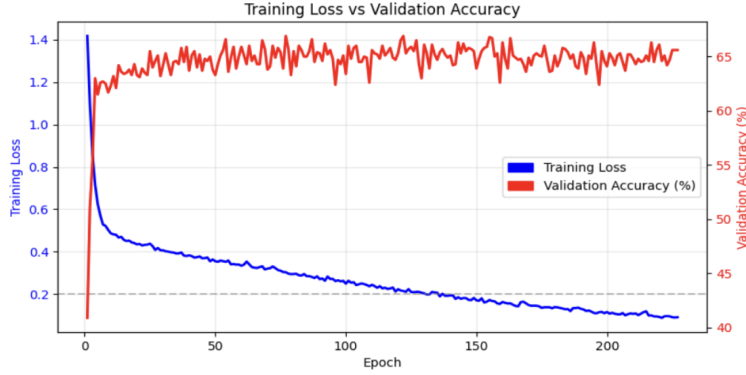
Layer	Parameters
Conv1 (2→16)	304
Conv2 (16→32)	4,640
Conv3 (32→16)	4,624
BatchNorm Layers	128
Fully Connected (256→58)	14,906
Final BatchNorm	116
Total	24,718

Inference and Size. At test time, we apply test-time augmentation (TTA) using flips and brightness perturbations and aggregate scores across multiple views. TTA improves robustness by averaging predictions across transformed inputs, reducing variance without increasing model complexity. The final model has **24,718** trainable parameters.

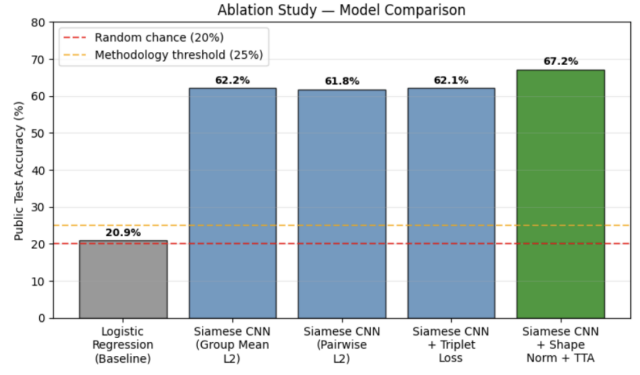
Key Design Choices.

- Siamese architecture for relational reasoning
- Distance-based outlier detection instead of classifier head
- Shape normalization to remove irrelevant variation
- Sobel edge augmentation to emphasize boundaries
- Test-time augmentation for robustness

Results and Analysis



(a) Training loss and validation accuracy.



(b) Ablation study across model variants.

Figure 1: Optimization and ablation results. Validation accuracy stabilizes in the mid-60% range, and the largest gain comes from moving to a distance-based Siamese formulation and then adding shape normalization.

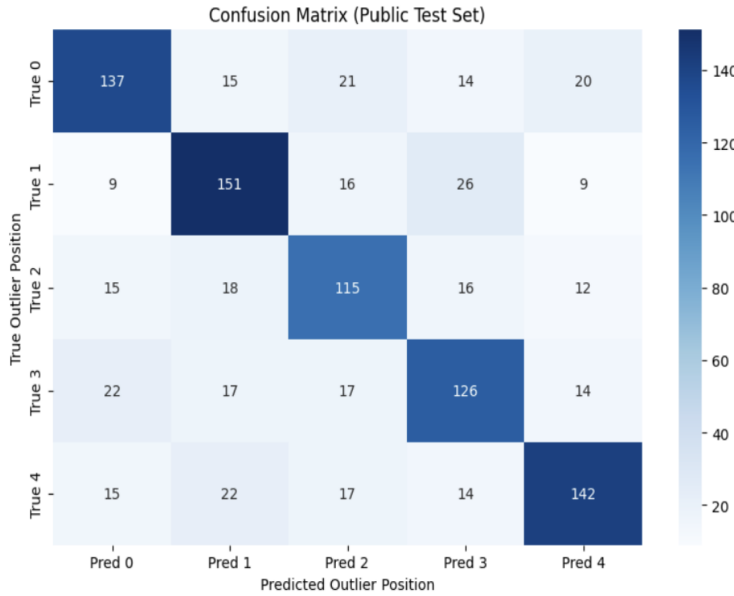
Training Dynamics. Figure 1(a) shows stable optimization: training loss decreases steadily, while validation accuracy quickly rises and remains between roughly 65–67%. The best validation score is 66.90%, which improves to 67.10% after TTA.

Ablation Study. Figure 1(b) summarizes the effect of key design choices. Each modification isolates a design component. The largest gain comes from replacing the classifier head with distance-based reasoning, followed by shape normalization, which significantly improves invariance to position and scale. The classifier head severely overfit, with training loss approaching zero while validation accuracy remained near random (21–23%), indicating memorization rather than learning meaningful relational structure.

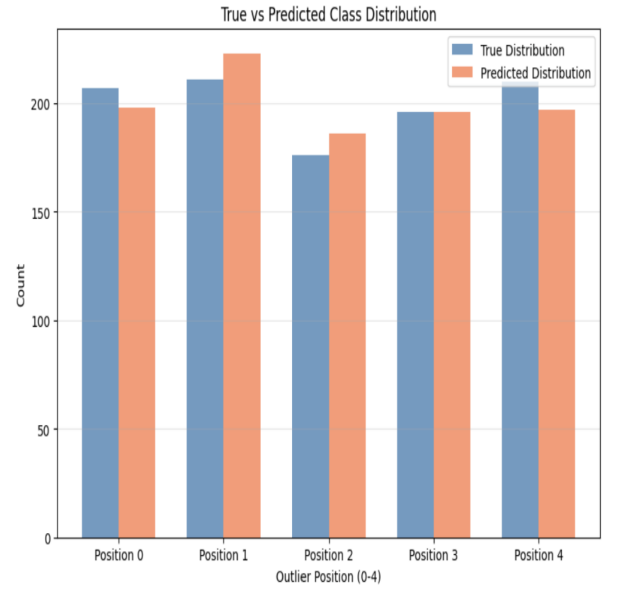
Adding handcrafted statistical features degraded performance (54–59%), suggesting that shape information is not well captured by simple global descriptors.

Model Variant	Public Accuracy (%)
Logistic Regression Baseline	20.9
Siamese CNN + Classifier Head	23–25
Hybrid + Handcrafted Features	54–59
Siamese CNN (Group Mean L2)	62.2
Siamese CNN (Average Pairwise L2)	61.8
Siamese CNN + Triplet Loss	62.1
Final: + Shape Norm + Sobel + TTA	67.1

Leaderboard Performance. Our final public Kaggle score is **67.10%**, well above both the random 20% baseline and the 25% methodology threshold.



(a) Confusion matrix on the public set.



(b) True vs. predicted class distribution.

Figure 2: Public-set error analysis. The confusion matrix is diagonally dominant, and the predicted class frequencies remain close to the true distribution, indicating limited positional bias.

Error Patterns. The confusion matrix shows consistent performance across all five positions, with the diagonal dominant in every row. The class-distribution plot also suggests the model does not exploit a strong positional shortcut. Most errors arise when shape differences are subtle or ambiguous.

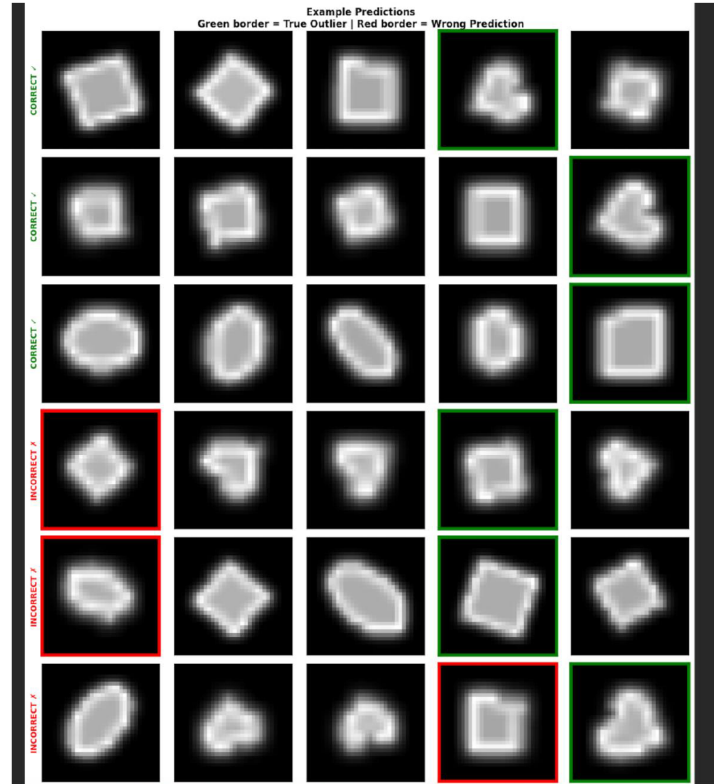


Figure 3: Qualitative examples from the public set. Green borders indicate correct outlier predictions and red borders indicate incorrect predictions. Failures often occur when the outlier differs only slightly from the four matching shapes.

Discussion and Conclusion

The strongest improvement comes from **shape normalization**, which removes irrelevant position and scale variation and lets the encoder focus on the actual shape class. The second key decision is the **distance-based**

Siamese formulation, which matches the relational nature of the problem and avoids the overfitting observed with classifier heads. Sobel edges and TTA provide smaller but consistent gains.

A limitation is that the labeled public test set is used for validation and model selection, which may make the reported public result slightly optimistic relative to the private score. The model is also constrained by the small dataset and strict parameter budget. Additionally, several values in our pipeline are fixed rather than tuned, notably the bounding box padding of 2 pixels and the threshold of 0.05 used in shape normalization. These were chosen by intuition and never validated through a systematic search, meaning the model’s performance depends on these hardcoded choices and may not generalize optimally to shape configurations outside the training distribution. Despite these limitations, the final system shows that careful preprocessing and problem-aligned inductive bias can outperform more complex alternatives in a low-resource regime.

We also experimented with cosine annealing learning rate schedules, which introduced instability and slightly reduced validation accuracy, and were therefore not used in the final model.

In summary, we present a compact Siamese CNN that achieves **67.10%** public accuracy with only **24,718** parameters. The results demonstrate that encoding the structure of the task directly into the model and preprocessing pipeline is highly effective for odd-one-out visual reasoning.

Future improvements could include self-supervised pretraining (e.g., contrastive learning such as SimCLR) to improve representation quality under limited data, as well as ensembling multiple models trained with different random seeds to reduce variance. Additionally, incorporating explicit shape descriptors such as Hu or Zernike moments may further improve performance.

Statement of Contributions

All team members wrote their own codes and the one with the best accuracy was selected. The report was written by Uday Parmar, Shahmeer Sajid and reviewed by Danyal Valika.

Statement on the Use of LLMs

Large language models were used for debugging implementation errors, guidance on report structuring and LaTeX formatting. The LLM was not used to implement the core algorithms. No code or data was shared with other teams.